Web : Apache, Nginx et HTTPS

BTS SIO 2e année SISR

Projet fil rouge · à partir du lundi 2 novembre 2026 · Blocs 2 et 3

Service web du laboratoire

Objectifs

- Publier un site avec un hôte virtuel Apache et le vérifier.
- Placer Nginx en frontal (proxy inverse) devant Apache, puis expliquer les journaux.
- Faire signer un certificat par l'autorité du laboratoire et passer le site en HTTPS.
- Prouver ce que HTTPS garantit, et ce qu'il ne garantit pas.

Prérequis

- VM Debian du laboratoire : web, client, éventuellement DNS ; SSH et `sudo`.
- Lien nom → adresse (DNS) ; requête et réponse HTTP, codes 200 et 404.
- Chiffrement asymétrique, empreinte et signature (Bloc 3, P01 intégrité et signature).

Sommaire

Étape	Travail
1	Identifier les rôles et l'état de départ
2	Apache : un hôte virtuel en HTTP
3	Nginx en frontal devant Apache
4	Une autorité de certification pour le laboratoire
5	HTTPS sur Nginx, redirection et preuves
6	Livrable, restitution et repli documentaire

Votre mission

Le laboratoire doit servir une page web interne, `web.lab.test`. Vous commencez par la configuration la plus classique, un **hôte virtuel Apache**. Vous placez ensuite **Nginx devant Apache** : c'est l'architecture d'une grande partie des sites réels, où un serveur frontal reçoit les clients et transmet au serveur d'application. Enfin, vous chiffrez l'accès en **HTTPS** avec un certificat signé par l'autorité de certification du laboratoire.

En binôme, alternez administration et contrôle. Chacun rédige sa propre conclusion. Travaillez exclusivement sur les VM attribuées, sans données réelles ni mot de passe réutilisé ailleurs. Le TP se poursuit sur plusieurs créneaux : conservez vos VM et complétez votre tableau de preuves à chaque étape.

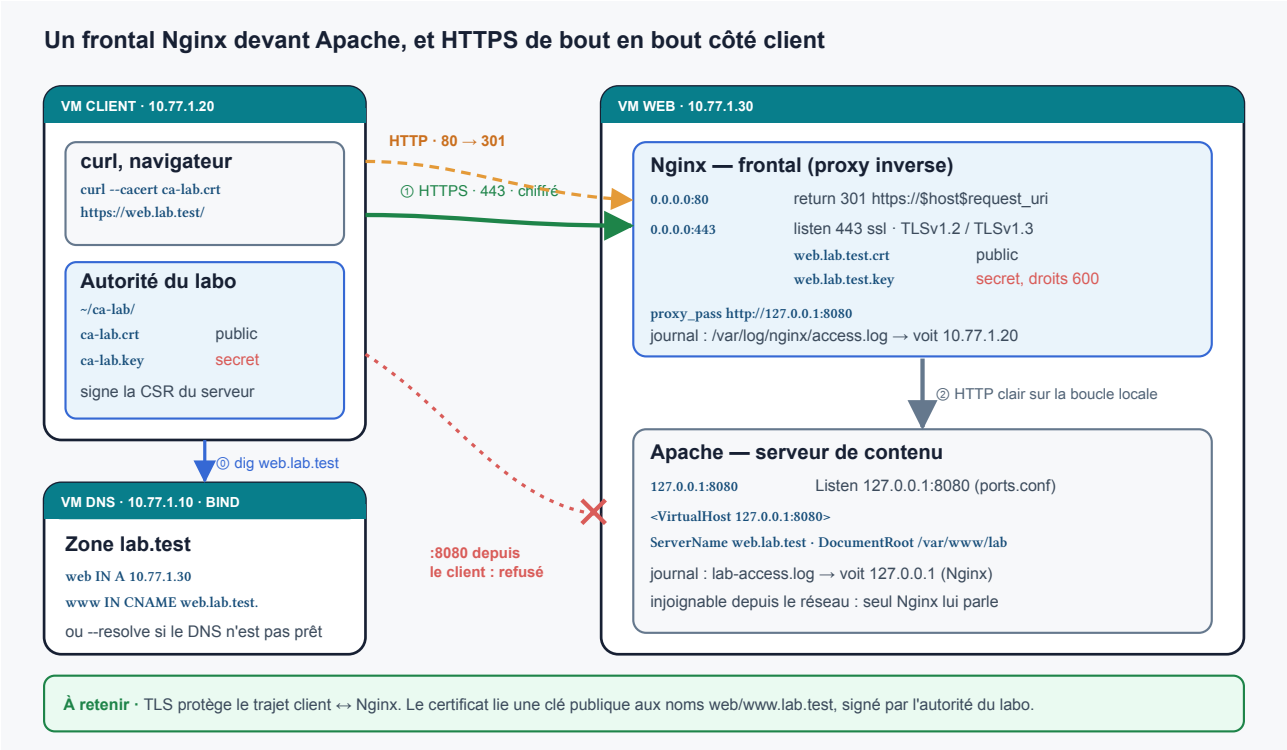


Fig. 1 - Architecture visée : le client interroge le DNS, puis joint Nginx en HTTPS ; Nginx transmet à Apache, qui n'écoute que sur 127.0.0.1:8080.

1. Identifier les rôles et l'état de départ

Les valeurs ci-dessous sont un exemple à remplacer par celles de votre binôme : DNS `10.77.1.10`, client `10.77.1.20`, web `10.77.1.30`, réseau `10.77.1.0/24`, zone `lab.test`. Si vous n'avez que deux VM, l'enseignant peut autoriser le rôle web sur une VM existante : notez alors son adresse réelle.

Sur la VM web :

```
hostname; ip -br address; ip route
dpkg -l apache2 nginx openssl | grep '^ii'
sudo ss -ltnp
```

Les paquets sont préinstallés par l'enseignant ; le réseau du laboratoire reste isolé. N'ouvrez pas d'accès Internet. `ss -ltnp` indique quel programme écoute sur quel port : **un seul service peut écouter sur un couple adresse/port**.

Question 1. Quels services écoutent sur le port 80 de la VM web ? Si Nginx et Apache sont tous deux actifs, que se passe-t-il au démarrage du second ?

Si Nginx occupe déjà le port 80, arrêtez-le pour l'étape 2 : `sudo systemctl stop nginx`.

2. Apache : un hôte virtuel en HTTP

Créez la page servie. Ce fichier est l'objet du TP : une page fictive, sans donnée réelle.

```
sudo mkdir -p /var/www/lab
sudo nano /var/www/lab/index.html
```

Rédigez un titre, la mention « Laboratoire SISR » et un identifiant fictif de binôme (exemple : `B07`). Puis créez l'hôte virtuel `/etc/apache2/sites-available/lab.conf` :

```
<VirtualHost *:80>
    ServerName web.lab.test
    ServerAlias www.lab.test
    DocumentRoot /var/www/lab

    ErrorLog ${APACHE_LOG_DIR}/lab-error.log
    CustomLog ${APACHE_LOG_DIR}/lab-access.log combined
</VirtualHost>
```

Activez le site, **validez avant de recharger**, puis vérifiez l'état du service :

```
sudo a2ensite lab
sudo apache2ctl configtest
sudo systemctl reload apache2
sudo apache2ctl -S
```

L'avertissement `AH00558 … fully qualified domain name` est sans conséquence ici : il signale seulement l'absence de `ServerName` global. `apache2ctl -S` liste les hôtes virtuels et celui qui sert par défaut. Conservez le site Debian `000-default` : c'est lui qui répond si le nom demandé ne correspond à aucun hôte.

Depuis le client, testez **sans dépendre du DNS** :

```
curl --noproxy '*' -i --resolve web.lab.test:80:10.77.1.30 \
  http://web.lab.test/
curl --noproxy '*' -i http://10.77.1.30/
curl --noproxy '*' -i --resolve web.lab.test:80:10.77.1.30 \
  http://web.lab.test/absent
```

Attendu : votre marqueur, puis la page Debian par défaut, puis `404 Not Found`. `--resolve` donne l'adresse à curl tout en envoyant le nom dans l'en-tête `Host`. Sur la VM web, rapprochez vos requêtes du journal :

```
sudo tail -n 5 /var/log/apache2/lab-access.log
```

Question 2. Pourquoi la requête vers l'adresse IP seule n'affiche-t-elle pas votre site ? Qu'est-ce qui permet à Apache de choisir l'hôte virtuel ?

Question 3. Le `404` prouve-t-il une panne réseau ? Que prouve-t-il ?

> **Repère**
>
> Si le DNS du laboratoire est opérationnel, publiez `web` (A) et `www` (CNAME vers `web.lab.test.`) dans la zone, incrémentez la série, validez avec `named-checkzone` puis vérifiez avec `dig @10.77.1.10 web.lab.test +norecurse`. Le TP peut se faire entièrement avec `--resolve` si le DNS n'est pas prêt.

3. Nginx en frontal devant Apache

Apache ne doit plus recevoir les clients directement : il n'écoutera que sur la boucle locale, port 8080. Sauvegardez avant de modifier (le suffixe ne doit pas déjà exister) :

```
sudo cp -n /etc/apache2/ports.conf /etc/apache2/ports.conf.avant-nginx
sudo cp -n /etc/apache2/sites-available/lab.conf \
  /etc/apache2/sites-available/lab.conf.avant-nginx
```

Dans `/etc/apache2/ports.conf`, remplacez `Listen 80` par `Listen 127.0.0.1:8080`. Dans `lab.conf`, remplacez `<VirtualHost *:80>` par `<VirtualHost 127.0.0.1:8080>`. Désactivez le site par défaut d'Apache, qui écoutait sur le port 80 :

```
sudo a2dissite 000-default
sudo apache2ctl configtest
sudo systemctl restart apache2
sudo ss -ltnp | grep -E ':80 |:8080 '
```

Attendu : Apache sur `127.0.0.1:8080`, plus rien sur le port 80. Un rechargement ne suffit pas pour changer de port d'écoute : il faut redémarrer.

Créez le site Nginx `/etc/nginx/sites-available/lab` :

```
server {
    listen 80;
    server_name web.lab.test www.lab.test;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Activez-le, sans écraser un lien existant, puis validez et démarrez :

```
sudo ln -s /etc/nginx/sites-available/lab /etc/nginx/sites-enabled/lab
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

Depuis le client, refaites le premier test de l'étape 2, puis observez les en-têtes :

```
curl --noproxy '*' -I --resolve web.lab.test:80:10.77.1.30 \
  http://web.lab.test/
```

Sur la VM web, comparez les deux journaux pour **la même requête** :

```
sudo tail -n 3 /var/log/nginx/access.log
sudo tail -n 3 /var/log/apache2/lab-access.log
```

Question 4. Quel serveur annonce l'en-tête `Server` ? Quelle adresse cliente apparaît dans chaque journal ? Expliquez la différence.

Question 5. Depuis le client, `curl http://10.77.1.30:8080/` aboutit-il ? Pourquoi est-ce souhaitable ?

Question 6. Citez deux intérêts d'un serveur frontal devant le serveur d'application.

Pour les rapides : faites journaliser l'adresse réelle du client par Apache avec le module `remoteip` et l'en-tête `X-Real-IP`. Vérifiez dans `lab-access.log`.

4. Une autorité de certification pour le laboratoire

Un navigateur fait confiance à un certificat s'il est **signé par une autorité** qu'il connaît déjà. Le laboratoire n'a pas d'autorité publique : vous en créez une, privée, dont le certificat sera distribué aux clients du laboratoire. L'autorité est installée sur la **VM client** : sa clé privée ne doit jamais se trouver sur le serveur web.

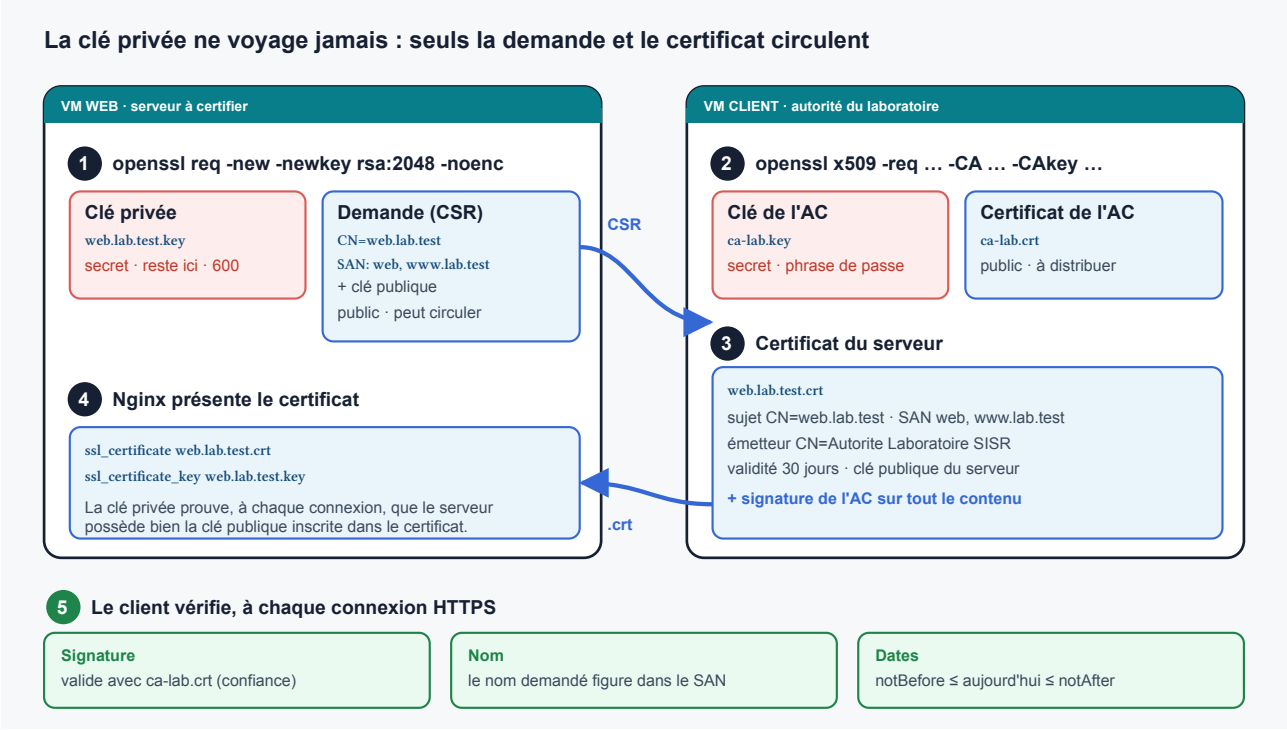


Fig. 2 - La clé privée du serveur reste sur la VM web : seule la demande de signature part vers l'autorité, qui renvoie le certificat signé.

Sur la VM client, créez l'autorité. La clé de l'autorité est protégée par une phrase de passe propre au laboratoire :

```
mkdir -p ~/ca-lab && cd ~/ca-lab
openssl req -x509 -newkey rsa:3072 -days 30 \
  -keyout ca-lab.key -out ca-lab.crt \
  -subj "/CN=Autorite Laboratoire SISR B07"
chmod 600 ca-lab.key
```

Sur la VM web, générez la clé du serveur et une **demande de signature** (CSR). La clé n'est pas chiffrée : Nginx doit pouvoir démarrer sans opérateur, elle est donc protégée par les droits du fichier.

```
cd ~
openssl req -new -newkey rsa:2048 -noenc \
  -keyout web.lab.test.key -out web.lab.test.csr \
  -subj "/CN=web.lab.test" \
  -addext "subjectAltName=DNS:web.lab.test,DNS:www.lab.test"
chmod 600 web.lab.test.key
openssl req -in web.lab.test.csr -noout -text | grep -A1 'Alternative'
```

Transférez **uniquement la CSR** vers le client (`scp`, ou copier-coller du texte PEM : une CSR est publique). La clé du serveur ne quitte pas la VM web. Sur le client, l'autorité signe :

```
cd ~/ca-lab
openssl x509 -req -in web.lab.test.csr -days 30 \
  -CA ca-lab.crt -CAkey ca-lab.key -CAcreateserial \
  -copy_extensions copy -out web.lab.test.crt
openssl x509 -in web.lab.test.crt -noout \
  -subject -issuer -dates -ext subjectAltName
openssl verify -CAfile ca-lab.crt web.lab.test.crt
```

Rapportez `web.lab.test.crt` sur la VM web (fichier public lui aussi).

Question 7. Complétez le tableau : quel fichier est secret, quel fichier est public, sur quelle VM se trouve-t-il ?

Fichier	Contenu	Secret ou public	VM
`ca-lab.key`	…	…	…
`ca-lab.crt`	…	…	…
`web.lab.test.key`	…	…	…
`web.lab.test.csr`	…	…	…
`web.lab.test.crt`	…	…	…

Question 8. Pourquoi le certificat porte-t-il les noms dans l'extension `subjectAltName`, et pas seulement dans le `CN` ?

5. HTTPS sur Nginx, redirection et preuves

Sur la VM web, installez la clé et le certificat avec des droits stricts :

```
sudo install -m 644 web.lab.test.crt /etc/ssl/certs/web.lab.test.crt
sudo install -m 600 web.lab.test.key /etc/ssl/private/web.lab.test.key
```

Sauvegardez `/etc/nginx/sites-available/lab`, puis remplacez son contenu par deux blocs : le port 80 ne fait plus que **rediriger** vers HTTPS.

```
server {
    listen 80;
    server_name web.lab.test www.lab.test;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name web.lab.test www.lab.test;

    ssl_certificate     /etc/ssl/certs/web.lab.test.crt;
    ssl_certificate_key /etc/ssl/private/web.lab.test.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo nginx -t && sudo systemctl reload nginx
sudo ss -ltnp | grep nginx
```

Depuis le client, menez les tests **dans cet ordre** et notez chaque résultat :

```
R80="web.lab.test:80:10.77.1.30"
R443="web.lab.test:443:10.77.1.30"
curl --noproxy '*' -I --resolve $R80 http://web.lab.test/
curl --noproxy '*' -i --resolve $R443 https://web.lab.test/
curl --noproxy '*' -i --cacert ~/ca-lab/ca-lab.crt \
  --resolve $R443 https://web.lab.test/
```

Attendu : `301` avec un en-tête `Location: https://…` ; puis un **échec de vérification** du certificat ; puis `200 OK` et votre marqueur. Observez enfin la négociation TLS :

```
openssl s_client -connect 10.77.1.30:443 -servername web.lab.test \
  -CAfile ~/ca-lab/ca-lab.crt < /dev/null 2>&1 \
  | grep -E 'subject=|issuer=|Verify return|Protocol|Cipher'
```

Un contrôle négatif, à réaliser et à expliquer :

```
curl --noproxy '*' -I --cacert ~/ca-lab/ca-lab.crt \
  --resolve autre.lab.test:443:10.77.1.30 https://autre.lab.test/
```

Question 9. Pourquoi le deuxième test échoue-t-il alors que la connexion est bien établie ? Que faudrait-il faire pour que les navigateurs des postes du laboratoire acceptent le site sans alerte ?

Question 10. Expliquez le contrôle négatif : quelle vérification a échoué, et pourquoi est-ce le comportement souhaité ? Quel protocole TLS a été négocié avec `s_client`, et quelle ligne de la configuration Nginx limite les versions acceptées ?

Question 11. Entre Nginx et Apache, sur `127.0.0.1:8080`, le trafic est-il chiffré ? Est-ce acceptable ici ? Dans quel cas ne le serait-il plus ?

Repères pour interpréter une erreur :

Observation	Cause probable	Premier contrôle
`Connection refused` sur 443	Nginx n'écoute pas en HTTPS	`nginx -t`, `ss -ltnp`, bloc `listen 443 ssl`
`unable to get local issuer certificate`	Autorité inconnue du client	Option `--cacert`, magasin de certificats
`no alternative certificate subject name matches`	Nom demandé absent du certificat	`-ext subjectAltName`, nom utilisé
`certificate has expired`	Date dépassée ou horloge fausse	`-dates`, `date` sur les deux VM
`502 Bad Gateway`	Nginx répond, Apache non joignable	`systemctl status apache2`, `ss` sur 8080

Pour les rapides : ajoutez l'en-tête `Strict-Transport-Security` (HSTS) avec une durée courte et expliquez le risque qu'il évite, puis ce qu'il impose en cas d'erreur de certificat.

6. Livrable, restitution et repli documentaire

Rendez un dossier court : schéma des rôles, adresses et ports (client ⟶ Nginx 80/443 ⟶ Apache 127.0.0.1:8080) ; configurations Apache et Nginx ; tableau **commande / attendu / observé / conclusion** ; extraits de journaux reliés à vos requêtes ; réponses aux questions. Aucune clé privée, phrase de passe ou donnée personnelle dans le dossier : seuls les certificats et les CSR, publics, peuvent y figurer.

Chaque étudiant rédige une conclusion de **6 à 8 lignes** : ce que l'accès HTTPS au laboratoire garantit (confidentialité, intégrité, authentification du serveur), ce qu'il ne garantit pas, et la preuve qui appuie chaque affirmation. Signalez les contrôles faits sur documents.

Réussite : `200 OK` via Nginx avec le marqueur du binôme, Apache inaccessible depuis le client, `301` vers HTTPS, `200 OK` en HTTPS avec `--cacert`, échec argumenté sans `--cacert` et sur un nom absent du certificat.

Repli documentaire

Ces sorties **fictives** permettent de répondre aux questions si une VM n'est pas prête. Intitulez alors votre conclusion « analyse sur documents » et n'attribuez pas ces sorties à vos VM.

```
$ sudo ss -ltnp
LISTEN 0.0.0.0:80       users:(("nginx",pid=812))
LISTEN 0.0.0.0:443      users:(("nginx",pid=812))
LISTEN 127.0.0.1:8080   users:(("apache2",pid=640))

$ curl -I --resolve web.lab.test:80:10.77.1.30 http://web.lab.test/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.22.1
Location: https://web.lab.test/

$ curl -i --resolve web.lab.test:443:10.77.1.30 https://web.lab.test/
curl: (60) SSL certificate problem: unable to get local issuer certificate

$ curl -i --cacert ca-lab.crt --resolve ... https://web.lab.test/
HTTP/1.1 200 OK
Server: nginx/1.22.1
<h1>Laboratoire SISR — binôme B07</h1>
```

```
nginx  access.log : 10.77.1.20 - - "GET / HTTP/1.1" 200 312
apache lab-access : 127.0.0.1 - - "GET / HTTP/1.0" 200 312

$ openssl s_client ... | grep ...
subject=CN = web.lab.test
issuer=CN = Autorite Laboratoire SISR B07
Protocol  : TLSv1.3
Cipher    : TLS_AES_256_GCM_SHA384
Verify return code: 0 (ok)
```

Repères

HTTP transporte la page ; TLS chiffre et authentifie le canal ; le certificat lie une clé publique à des noms, sous la signature d'une autorité. Références : Apache, hôtes virtuels, Nginx, proxy inverse, Nginx, HTTPS, OpenSSL, req.

L'IA peut expliquer un message d'erreur après une première analyse personnelle. Vérifiez toute commande avant de l'exécuter, ne transmettez ni clé privée ni phrase de passe, et rédigez votre conclusion vous-même.